Project 6

Project Name:

Web-based Supply Chain Management system for a company (WBSCM)

Team Members:

- Arnib Alam Farooqui
- Daniel Hernandez

Work Done:

Over the past couple weeks, we have shifted a large amount of our focus to back end and implementation of OOAD patterns within this code. We wrote large chunks of code which were as close to our class diagram as possible. We have implemented classes such as payment strategies, carts and customers etc. Another large portion of our focus was our database integration. We are still having some issues with said integration and getting everything to work and be connected correctly. Created basic db tables for product, supplier, orders and users. We are using MySql to create a db and setup server. We are looking at things like docker and postman got connections in integration.

Patterns Usage:

1. **Abstract Factory Pattern with Products creation**

The Abstract Factory Pattern provides an interface for creating families of related objects without specifying their concrete classes. This pattern can be applied to a product-creation mechanism in our system. AbstractProductFactory interface can define a method createProduct(), and concrete factories like ElectronicProductFactory and FurnitureProductFactory will create and return an instance of ElectronicProduct and FurnitureProduct respectively. This way, the system can produce different types of products through a common interface, which improves the modularity and flexibility of the code.

2. **Observer Pattern with Order**

The Observer Pattern defines a one-to-many dependency between objects, so when one object changes state, all its dependents are notified automatically. This is useful in our system where we have Order objects whose state may change over time.

3. **Strategy Pattern with Payment**

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. This is a perfect fit in our system for different payment methods. PaymentStrategy is an interface that declares a method executePayment(). This interface is implemented by multiple concrete strategy classes like CreditCardStrategy and PaypalStrategy, each providing a different payment method. An Order object maintains a reference to a PaymentStrategy object and can execute a payment using the current payment strategy, providing flexibility to switch payment methods at runtime.

4. **Singleton Pattern with User Authentication**

The Singleton pattern is employed in the 'UserAuthentication' class to ensure that there is a single, globally accessible instance managing user authentication throughout the application. By restricting the instantiation of the class to a single instance and providing a global access point via the getInstance() method, the pattern promotes consistency in handling authentication state across different components.

5. **Model-View-Controller (MVC) Pattern with ApiController**

The 'ApiController' class, as presented in the context of a RESTful API in a Spring Boot application, aligns with the principles of the Controller component in the MVC design pattern.

In the MVC pattern:

- Model: Represents the data and business logic of the application. In your case, this would include entities such as Product, Order, and associated services.
- View: Represents the presentation layer responsible for displaying the user interface. In a RESTful API, the view is often abstracted away as the data is typically returned in a structured format (like JSON) and doesn't directly deal with rendering views.
- Controller: Acts as an intermediary between the Model and the View. It receives user input, processes it (possibly invoking Model services), and returns the appropriate response. The ApiController class, in this context, handles incoming HTTP requests, delegates the processing to appropriate services, and returns HTTP responses.

Changes:

In contrast to the initial design in Project 5, our current approach involves creating a Spring application to seamlessly access the MySQL Database. Leveraging Spring Initializr streamlines the setup process by pulling in all the necessary dependencies for our application. With this foundation, we now turn to Spring Data JPA to handle the intricacies of database interactions. Spring Data JPA repositories are interfaces equipped with methods that facilitate the creation, reading, updating, and deletion of records against the backend data store. To test the REST API calls, we will use API development tools such as Postman software to send and receive requests, HTTP(POST,GET or DELETE) data to the server.

Issues/Challenges:

The primary challenge moving forward lies in orchestrating the integration of various components, connecting the database to both the backend and frontend. This integration is crucial to ensure that PUT and POST requests are accurately handled and reflect up-to-date information. Furthermore, the potential evolution of the database schema, especially during development, may require corresponding adjustments to the application.The goal is to establish a cohesive system where data flows seamlessly between the different layers of the application, presenting accurate and synchronized information to users.

Plan for Next Iteration

Going forward within our project we will see a shift in focus to the database and front end development. We understand that implementation of a front end with our already extensive back end will be a lot so we will keep the front end simple and minimalistic since most of our OOAD principles are shown off in the back end. We want to have a basic front end that shows users and a process of them ordering things. A typical path that a user would take when using a WBSCM. Basic UI for users to navigate. We will obviously want a successful connection to the back end with updated put and post requests.

Here's the status summary of the Project's component with reference to the plan of our next iteration:

<markdown>

Task (Status: Finish)	Task (Status: Plan To start)
Backend (Java Classes with design Patterns)	Front-End Wireframes and Mockups
Database Creation in MySQL	Front-End Foundation
Building REST services with Spring	Back-End to Front-End Connection
	Product Display on Front End

	Ordering Process Implementation
	Responsive Design
	Front-End Testing and Iteration